# SAPO_Dockeract

Docker Containerization Activity — IPT 102 Integrative Programming and Technologies 2

Student: SAPO | Program/Section: BSIT | Date: October 22, 2026

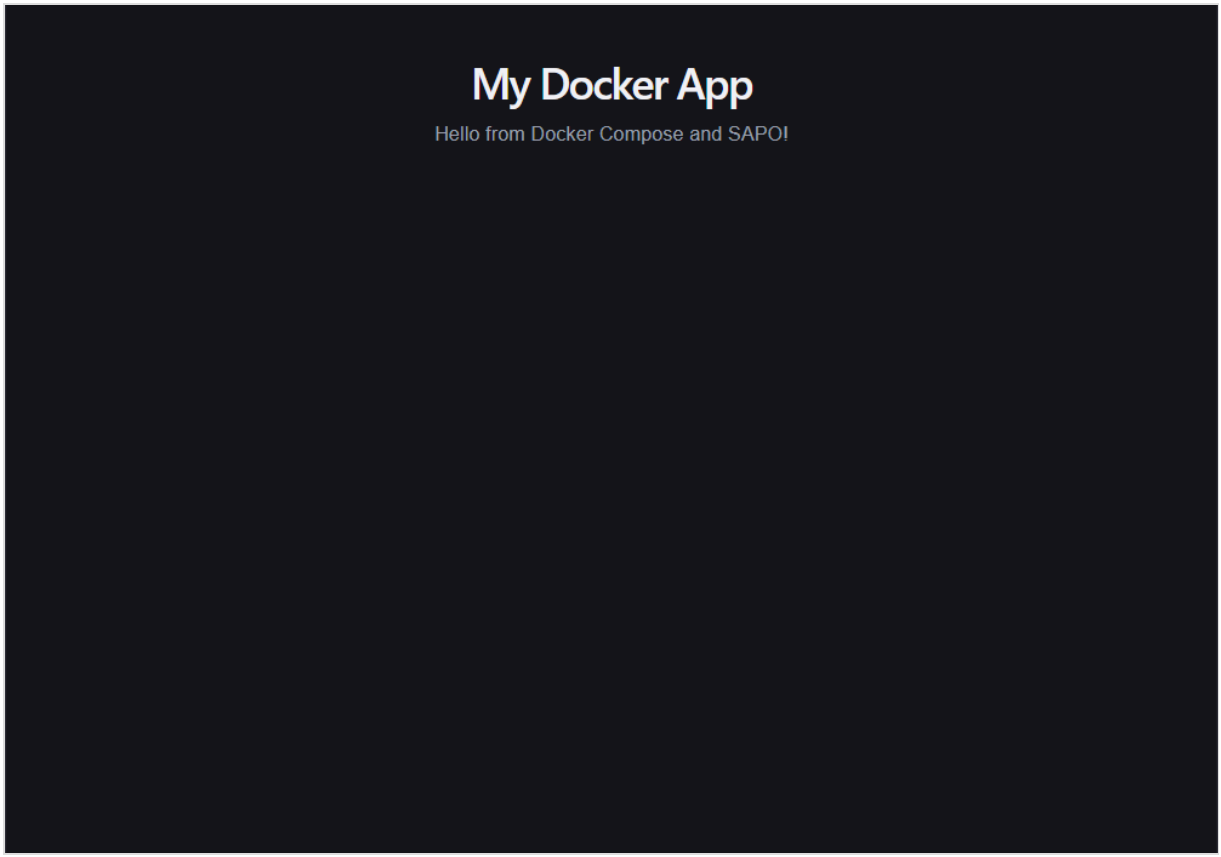
Faculty: Jeremy Zion L. Jamer

Deliverable: built web app + answered task reflections (paper work).

## 1. Built web app

Built **myapp**: a React (Vite) frontend served by Nginx, a .NET 10 Web API, and PostgreSQL 16, orchestrated with Docker Compose.

| File | Purpose |
|---|---|
| api/ | .NET 10 Web API with GET /api/message and CORS |
| web/ | React + Vite frontend calling the API |
| docker-compose.yml | api, web, and db services |
| .env | APP_MESSAGE and DB_PASSWORD |



## Reflections — Step 1

> **1.** In your own words, what is Docker? What problem does it solve?

Docker is a platform that packages an application with everything it needs into images and runs them as containers. It solves environment inconsistency so the same app runs identically on any machine instead of breaking with “works on my machine.”

## Reflections — Step 2

> **2.** Did you have any problem during installation? Write the problem and how you fixed it. If none, write how you checked that Docker works.

No installation problem. Docker Desktop installed successfully; checked with <code>docker --version</code> (29.6.2) and <code>docker run hello-world</code>, which printed “Hello from Docker!”, confirming Docker works.

## Reflections — Step 3

> **3.** If we will run the app in containers later, why do we first install Node.js and .NET on our own computer?

To create, build, and test the application locally before moving it into containers. Containers still use SDK/runtime base images, but local tools let us develop and verify the app first.

## Reflections — Step 4

> **4.** Write the versions you installed.

Node.js: v24.19.0   .NET: 10.0.402   Docker: 29.6.2

## Reflections — Step 5

> **5.** In your own words, what does the endpoint /api/message do?

It returns a JSON object { “message”: “…” } from the API. The text comes from the Message setting and defaults to “Hello from the API”.

## Reflections — Step 6

> **6.** Why did we add CORS to the code? What can happen if we do not add it?

CORS lets the React frontend on another origin call the API. Without it, the browser blocks cross-origin requests and shows “blocked by CORS policy.”

## Reflections — Step 7

> **7.** What is the job of the frontend? What is the job of the API? How do they talk to each other?

The frontend shows the UI and fetches/displays the message; the API provides the endpoint and JSON response. They talk over HTTP: React calls ${API_URL}/api/message.

## Reflections — Step 8

> **8.** Right now your app only works if Node.js and .NET are installed. What problem can happen when you give this app to a classmate?

If the classmate lacks Node.js/.NET or has wrong versions, they cannot build or run the app. Containers solve this by packaging all dependencies into images.

## Reflections — Step 9

> **9.** What is the difference between an image and a container? Give your own example.

An image is a read-only blueprint; a container is a running copy of that image. Example: myapp-api:1.0 is the image; myapp-api is the running container.

## Reflections — Step 10

> **10.** Your Dockerfile has two FROM lines (two stages). Why do we use two stages?

Stage 1 builds/publishes with the SDK; stage 2 runs only the published output with the smaller runtime image. This keeps the final image small.

## Reflections — Step 11

> **11.** Why do we copy only the dist folder into the Nginx stage and not the whole project?

Nginx only needs the compiled static files. Copying only dist makes the runtime image smaller and excludes source, node_modules, and build tools.

## Reflections — Step 12

> **12.** Type docker images. Write the names and sizes of your two images. Which one is bigger? Why do you think so?

myapp-api:1.0 = 340 MB; myapp-web:1.0 = 93.9 MB. The API image is bigger because it contains the ASP.NET runtime, while the web image is mostly static files served by Nginx.

## Reflections — Step 13

> **13.** Write three things Docker Compose does for us that are harder with many docker run commands.

1) Starts all services with one command. 2) Declares builds, ports, networks, and volumes in one file. 3) Stops/removes everything together with one command.

## Reflections — Step 14

> **14.** What does depends_on do? What does it not do?

depends_on controls startup order so the API starts before the web container. It does not wait for the API to be ready to serve requests.

## Reflections — Step 15

> **15.** Why do we put values like passwords in a .env file and not directly in docker-compose.yml or the code?

So settings can change without editing code or Compose files, and secrets are not committed to version control.

## Reflections — Step 16

> **16.** How do you write the setting ConnectionStrings:Default as an environment variable in .NET?

ConnectionStrings__Default — colons become double underscores in .NET environment variables.

## Reflections — Step 17

> **17.** How does the web container find the API container? Why does the database have no ports line?

Through the Compose private network using the service name “api”. The database has no published port because only other containers should reach it, not the host.

## Reflections — Step 18

> **18.** What does localhost mean inside a container? Why is this a common mistake when connecting to a database?

Inside a container, localhost means that container itself, not the host. Connecting to the database via localhost fails because the database runs in the db container.

## Reflections — Step 19

> **19.** What happened to your data after docker compose down? What happened after docker compose down -v? Explain why.

After <code>down</code>, the row Juan was still there because only containers were removed; the db-data volume remained. After <code>down -v</code>, the volume was deleted too, so the students table no longer existed.

## Reflections — Step 20

> **20.** Think of a real system (for example a school enrollment system). Why is losing data a serious problem? How does a volume help?

Losing enrollment records or grades would corrupt the system. A volume keeps database files outside the container so data survives container removal and recreation.

## Reflections — Step 21

> **21.** List the three most important things you learned in this handout.

1) Build multi-stage Docker images. 2) Run related services together with Docker Compose and private networking. 3) Persist database data with named volumes.

## Reflections — Step 22

> **22.** Which step was the hardest for you? What made it hard, and how did you solve it?

Pulling the base images was the hardest; Docker Hub pulls were slow and timed out at first. Retrying the pulls resolved it, and all images downloaded successfully.

## Reflections — Step 23

> **23.** How can you use Docker in your Integrative Programming 2 finals project?

Containerize each project tier, run them together with Docker Compose, persist database data in volumes, and configure the app with environment variables for easy deployment.

## Verification evidence

| Check | Result |
|---|---|
| Tool versions | Docker 29.6.2 · .NET SDK 10.0.402 · Node v24.19.0 · npm 11.17.0 |
| Local API (dotnet run) | GET http://localhost:5050/api/message returned {"message":"Hello from the API"} |
| Local frontend (npm run dev) | Vite served http://localhost:5173, status 200 |
| Image sizes | myapp-api:1.0 = 340 MB · myapp-web:1.0 = 93.9 MB |
| Compose services | api, web, db all Up; network myapp_default present |
| Env config | Changing APP_MESSAGE in .env then `docker compose up -d` updated the API/page message |
| Service-name networking | `docker compose exec web ping -c 2 api` and db: 2 packets received each |
| Database | `psql -U postgres -d appdb -c "\l"` lists appdb |
| Volume persistence | After `down`/`up -d`, row Juan remained; after `down -v`/`up -d`, relation students did not exist |

Source: C:\\Users\\alexa\\OneDrive\\Documents\\docker act\\myapp (README.md, documentation.md, Dockerfiles, compose, .env included.)